



UNIVERSITY OF BUEA

REPUBLIC OF CAMEROON
PEACE - WORK - FATHERLAND

FACULTY OF ENGINEERING AND TECHNOLOGY DEPARTMENT OF COMPUTER ENGINEERING

ADAPTIS

AN ADAPTIVE, NETWORK-AWARE MOBILE E-LEARNING PLATFORM FOR LOW-CONNECTIVITY ENVIRONMENTS

A report submitted to the Department of Computer Engineering, Faculty of Engineering and Technology, University of Buea, in partial fulfilment of the requirements for the course.

Presented by: GROUP 18

S/N	NAMES	MATRICULE
1	DIVINE JEMBONG TABI NDIP	FE23A035
2	LENA KEZIAH ABANDA GHA	FE23A081
3	MENYE EWODO GERMAINE LAETICIA	FE23A092
4	NKONGHO ELIZABETH AGBOR-MANYO	FE23A122
5	TANDO CALVIN NKUH (Leader)	FE23A159

Course: CEF 440 - Internet and Mobile Programming

Course Instructor/ Supervisor:

Dr. Nkemeni Valery
University of Buea

2025/2026 Academic Year

CERTIFICATION OF ORIGINALITY

We, the undersigned, being the members of Group 18, hereby certify that this dissertation entitled “ADAPTIS: AN ADAPTIVE, NETWORK-AWARE MOBILE E-LEARNING PLATFORM FOR LOW-CONNECTIVITY ENVIRONMENTS” was carried out by us in the Department of Computer Engineering, Faculty of Engineering and Technology, University of Buea, under the supervision of Dr. Nkemeni Valery.

This dissertation is authentic and represents the fruit of our own collective research, design and implementation efforts, undertaken as part of the CEF 440 course project.

NKONGHO ELIZABETH AGBOR-MANYO (FE23A122)

TANDO CALVIN NKUH (FE23A159)

DIVINE JEMBONG TABI NDIP (FE23A035)

LENA KEZIAH ABANDA GHA (FE23A081)

MENYE EWODO GERMAINE LAETICIA (FE23A092)

Supervisor

Dr. Nkemeni Valery

Date: _____

Signature: _____

Head of Department

Date: _____

Signature: _____

DEDICATION

This work is dedicated to our families, whose continuous support and encouragement have sustained us throughout our studies, and to every student striving to learn under the constraint of an unreliable internet connection - the very challenge that inspired this project.

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our supervisor and course instructor, Dr. Nkemeni Valery, for the guidance, patience and constructive criticism offered throughout the design and implementation of this project. His feedback on our requirements specification, architecture and successive prototypes shaped the platform into what it has become.

Our gratitude also goes to the Department of Computer Engineering and the Faculty of Engineering and Technology, University of Buea, for providing the academic environment and resources that made this work possible.

We are grateful to the fifty-nine respondents who took part in our requirements-gathering survey, and to the students and teachers who tested early versions of Adaptis and offered feedback that directly shaped its features.

Finally, we thank our families and friends for their unwavering encouragement, understanding and support throughout the duration of this project.

ABSTRACT

Access to online education across Sub-Saharan Africa, and Cameroon in particular, is routinely disrupted by unstable internet connectivity, high mobile-data costs and predominantly low-end handsets. A requirements-gathering survey of fifty-nine respondents conducted as part of this study found that 95% experience poor or average connectivity while learning online, 97% encounter recurring network problems, and 90% have abandoned an online lesson altogether because of a failed connection. Existing platforms such as Moodle, Kolibri and Eneza Education each address only part of this problem: none combine real-time, Quality-of-Experience-driven content adaptation with offline-first storage and automatic progress synchronisation in a single mobile-first solution. This project set out to design and implement Adaptis, an adaptive, network-aware mobile e-learning platform that closes this gap.

The methodology combined an Agile, iterative development process with an empirical, survey-driven requirements phase. Gathered requirements were classified, prioritised using the MoSCoW technique and translated into a formal system design comprising context, data-flow, use-case, sequence, class and deployment diagrams, together with a MongoDB document schema built around a three-tier content model - high-definition video, standard-definition video, and an audio-and-text ‘Lite Mode’. The system was realised as a Flutter mobile application for students, a React web dashboard for teachers and administrators, and a Node.js/Express backend exposing a RESTful, JWT-secured API backed by MongoDB Atlas. The mobile client continuously evaluates network quality using the `connectivity_plus` package and switches automatically between the three content tiers, while an offline-first local store preserves learner progress and re-synchronises it once connectivity is restored.

The resulting platform was deployed for evaluation - the mobile application through Expo and the two web portals through Replit Deployments - and tested for registration, authentication, navigation, course access and cross-platform responsiveness. The results demonstrate that a mobile-first, three-tier adaptive delivery strategy paired with an offline-first data architecture can measurably improve learning continuity for students in low-resource, low-connectivity environments, while remaining light enough to run on modest hardware. The project contributes a working reference architecture for connectivity-resilient e-learning and identifies concrete directions - store publication, deeper analytics and multilingual support among them - for extending the platform beyond this first version.

Keywords: Adaptive e-learning, Quality of Experience (QoE), offline-first architecture, adaptive bitrate streaming, low-connectivity environments, mobile learning, MongoDB, Flutter.

TABLE OF CONTENTS

(Right-click and choose ‘Update Field’, then ‘Update entire table’ after any final edits, so that page numbers reflect the finished layout.)

CHAPTER ONE	1
GENERAL INTRODUCTION	1
1. Background and Context of the Study	1
2. Problem Statement	3
3. Objectives of the Study	3
3.1 General Objective.....	3
3.2 Specific Objectives.....	3
4. Proposed Methodology	4
5. Research Questions	4
6. Research Hypothesis	5
7. Significance of the Study	5
8. Scope of the Study.....	5
9. Delimitation of the Study	6
10. Definition of Keywords and Terms.....	6
11. Organization of the Dissertation	6
CHAPTER TWO.....	8
LITERATURE REVIEW.....	8
1. Introduction	8
2. General Concepts on Adaptive, Network-Aware E-Learning	8
2.1 Quality of Service versus Quality of Experience	8
2.2 Adaptive Bitrate Streaming.....	8
2.3 Offline-First Architecture and Reactive UI Patterns.....	8
2.4 Document-Oriented Data Storage	9
3. Related Works	9
4. The Research Gap	10
5. Partial Conclusion	10
CHAPTER THREE.....	11
ANALYSIS AND DESIGN.....	11
1. Introduction	11
2. Proposed Methodology	11
2.1 Requirements Engineering Process	11
2.2 Field Data Collection	12
2.3 Quantitative Findings	12
2.4 Requirement Classification and Prioritisation.....	12

2.5 Requirement Validation	13
2.6 Risk Assessment and Reluctance Mitigation	14
3. Design.....	14
3.1 Application Identity.....	14
3.2 Visual and Interaction Design.....	15
3.3 Conceptual Database Design.....	16
4. Global Architecture of the Solution	17
5. Description of the Resolution Process	18
5.1 Context Diagram	18
5.2 Data Flow Diagram	19
5.3 Use Case Diagram.....	20
5.4 Sequence Diagram.....	21
5.5 Class Diagram	22
5.6 Deployment Diagram	23
6. Partial Conclusion	24
CHAPTER FOUR	25
IMPLEMENTATION AND RESULTS	25
1. Introduction	25
2. Tools and Materials Used.....	25
3. Description of the Implementation Process	25
3.1 Student Mobile Application	25
3.2 Teacher and Administrator Web Dashboard.....	26
3.3 Backend API and Database	26
3.4 Authentication and the Adaptive Learning Engine	26
3.5 Deployment	27
4. Presentation and Interpretation of Results	27
4.1 Mobile Application	27
4.2 Administrator and Teacher Web Portals	29
4.3 Testing.....	31
5. Evaluation of the Solution.....	31
6. Partial Conclusion	32
CHAPTER FIVE.....	33
CONCLUSION AND FURTHER WORKS.....	33
1. Summary of Findings.....	33
2. Contribution to Engineering and Technology	33
3. Recommendations	34
4. Difficulties Encountered	34
5. Further Works	35
6. Conclusion.....	35

LIST OF TABLES

Table 1: Stakeholder Analysis Summary	11
Table 2: Key Survey Findings and their Influence on Requirements.....	12
Table 3: MoSCoW Prioritisation Criteria.....	13
Table 4: Reluctance Mitigation Summary.....	14
Table 5: Core Database Entities	16
Table 6: Technology Stack Summary	18

LIST OF FIGURES

Figure 1: Adaptis application logo and launcher-icon variants	15
Figure 2: Figma prototype screens (1 of 2) used to validate the visual and interaction design.....	15
Figure 3: Figma prototype screens (2 of 2) used to validate the visual and interaction design.....	16
Figure 4: Entity-Relationship Diagram of the Adaptis database	17
Figure 5: Context Diagram of the Adaptis platform.....	19
Figure 6: Level 1 Data Flow Diagram of the Adaptis platform	20
Figure 7: Use Case Diagram of the Adaptis platform	21
Figure 8: Sequence Diagram of a typical Adaptis session	22
Figure 9: Class Diagram of the Adaptis platform	23
Figure 10: Deployment Diagram of the Adaptis platform	24
Figure 11: Adaptis mobile application QR code and deployment access point	28
Figure 12: Mobile application login screens	28
Figure 13: Student mobile learning interface	28
Figure 14: Adaptive lesson delivery across content tiers (1 of 2)	29
Figure 15: Adaptive lesson delivery across content tiers (2 of 2)	29
Figure 16: Admin Portal dashboard screens (1 of 2).....	30
Figure 17: Admin Portal dashboard screens (2 of 2).....	30
Figure 18: Admin Portal analytics and user-management screens.....	30
Figure 19: Admin Portal system-monitoring screen	30
Figure 20: Teacher Portal course-management screens	30
Figure 21: Teacher Portal lesson and quiz-creation screens.....	31
Figure 22: Teacher Portal student-grading and messaging screens.....	31
Figure 23: Runtime issues observed and resolved during post-deployment testing.....	31

LIST OF ABBREVIATIONS

API	Application Programming Interface
AES	Advanced Encryption Standard
ARM	Advanced RISC Machine (processor architecture)
CDN	Content Delivery Network
FCM	Firebase Cloud Messaging
FR	Functional Requirement
HD	High Definition
HLS	HTTP Live Streaming
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
MoSCoW	Must have, Should have, Could have, Won't have (prioritisation technique)
MVC	Model-View-Controller
MVP	Model-View-Presenter
MVVM	Model-View-ViewModel
NFR	Non-Functional Requirement
QoE	Quality of Experience
QoS	Quality of Service
REST	Representational State Transfer
SD	Standard Definition
SDLC	System Development Life Cycle
SRS	Software Requirements Specification
UI/UX	User Interface / User Experience
UML	Unified Modeling Language

CHAPTER ONE

GENERAL INTRODUCTION

1. Background and Context of the Study

The rapid advancement of Information and Communication Technology (ICT) has significantly transformed the education sector by introducing e-learning as an effective alternative to traditional classroom-based learning. Through internet-enabled platforms, students can access educational resources, attend virtual lectures, complete assessments, and interact with instructors regardless of their geographical location. This flexibility has made e-learning an essential component of modern education, particularly following the increased demand for remote learning. Consequently, educational institutions across the world have adopted Learning Management Systems (LMS) such as Moodle, Google Classroom, and Canvas to improve access to quality education.

Despite these advancements, the effectiveness of e-learning largely depends on the availability of reliable internet connectivity and adequate digital infrastructure. In many developing countries, especially within Sub-Saharan Africa and Cameroon, these requirements are often difficult to meet. A considerable proportion of students rely on mobile data networks that are unstable, slow, and expensive (GSMA, 2023). Frequent network interruptions, high latency, limited bandwidth, and poor coverage negatively affect learners' ability to stream video lectures, download learning materials, participate in online assessments, and communicate effectively with instructors. Consequently, many learners experience frustration, reduced engagement, and in some cases discontinue online learning altogether.

Most existing e-learning platforms are designed with the assumption that users have stable and high-speed internet connections. Their primary mode of content delivery depends heavily on continuous video streaming, making them unsuitable for environments where internet quality fluctuates frequently. Although some platforms provide downloadable materials for offline use, they generally lack intelligent mechanisms that automatically adapt learning content according to the user's current network conditions. As a result, learners in low-resource settings continue to face significant barriers to accessing uninterrupted online education.

To better understand these challenges, a requirements-gathering survey involving fifty-nine (59) respondents was conducted as part of this project's Software Requirements

Specification (SRS) process. The survey findings revealed that approximately ninety-five percent (95%) of respondents experience poor or average internet connectivity during online learning, while ninety-seven percent (97%) reported encountering network problems either always or occasionally. Furthermore, eighty-three percent (83%) of respondents primarily use smartphones and mobile data to access learning materials, indicating that mobile-first and data-efficient solutions are essential. The survey also showed that ninety-two percent (92%) of respondents considered automatic progress saving an important feature, while eighty-six percent (86%) preferred the ability to download learning materials for offline access. Most importantly, about ninety percent (90%) acknowledged that poor internet connectivity had previously forced them to stop learning before completing their lessons. These findings clearly demonstrate that network reliability remains one of the greatest obstacles to inclusive online education.

Several existing educational platforms, such as Moodle, Kolibri, and Eneza Education, have attempted to address some of these challenges through features such as offline content access, lightweight learning materials, and mobile compatibility. However, each platform addresses only part of the problem. None effectively combines adaptive multimedia delivery, intelligent bandwidth monitoring, offline-first learning, automatic synchronisation of learning progress, and Quality of Experience (QoE)-driven content adaptation within a single integrated solution designed specifically for low-resource environments. This creates an opportunity to develop a more comprehensive platform that addresses the unique challenges faced by learners in developing regions.

Quality of Experience (QoE) has emerged as an important concept in the design of modern multimedia systems because it focuses on the overall satisfaction and perceived performance experienced by end users, rather than relying solely on technical Quality of Service (QoS) metrics such as bandwidth and latency (ITU-T, 2017). By continuously monitoring network conditions and dynamically adjusting content delivery according to available bandwidth, QoE-based systems can provide smoother learning experiences while minimising interruptions and unnecessary data consumption.

In response to these challenges, this project proposes the design and implementation of Adaptis, an adaptive, network-aware mobile e-learning platform for inclusive online education in low-resource settings. The proposed platform intelligently detects internet quality in real time and automatically switches between high-definition video, standard-definition video, and an audio-and-text 'Lite Mode' depending on current network conditions. Additionally, the system incorporates offline content storage, automatic synchronisation of learner progress when

connectivity is restored, secure user authentication, quizzes, assignment submission, and efficient course management. By combining adaptive content delivery with an offline-first architecture, the platform seeks to ensure continuous learning even under poor or unstable internet connectivity.

2. Problem Statement

Learners in low-resource environments such as Cameroon are systematically excluded from the benefits of online education because the platforms available to them assume a level of network stability that simply does not exist for most users. The evidence gathered for this project is unambiguous: 97% of surveyed respondents encounter network problems while learning online, and 90% have stopped learning altogether at some point because of a failed or degraded connection. Existing e-learning tools respond to this reality poorly - they either stream a single, fixed quality of media regardless of the user's actual network condition, or they provide only a coarse, manual toggle between an 'online' and an 'offline' mode that requires the learner to anticipate connectivity problems in advance. Neither approach responds to the fact that network quality on mobile networks in the region changes minute to minute, often several times within a single lesson.

The central problem addressed by this project is therefore the absence of a mobile e-learning platform that treats intermittent, low-bandwidth connectivity as its default operating condition rather than an exception, and that can automatically and transparently adapt the format of learning content - and preserve a learner's progress - as network conditions change, without requiring manual intervention from the student.

3. Objectives of the Study

3.1 General Objective

The general objective of this project is to design and implement Adaptis, an adaptive, network-aware mobile e-learning platform that maintains continuous learning access for students in low-connectivity, low-resource environments by automatically adjusting content delivery to real-time network conditions.

3.2 Specific Objectives

- To investigate, through a structured field survey, the connectivity constraints, device profiles and content preferences of prospective users of an e-learning platform in Cameroon.

- To review existing e-learning and adaptive-streaming solutions and identify the specific gap that a QoE-driven, offline-first mobile platform can fill.
- To gather, classify and prioritise functional and non-functional requirements for the platform using an Agile, survey-informed process.
- To design the system architecture, data model and adaptive content-delivery logic of the platform, expressed through context, data-flow, use-case, sequence, class and deployment diagrams.
- To implement a Flutter-based mobile application for students, a React-based web dashboard for teachers and administrators, and a Node.js/Express/MongoDB backend that together realise the adaptive, offline-first design.
- To deploy and evaluate the resulting platform against the objectives and requirements established in the early phases of the project.

4. Proposed Methodology

This project follows an Agile, iterative software development methodology structured around short sprints, combined with an empirical, survey-driven requirements-engineering phase. Primary data on user needs, device ownership and network conditions was collected through a structured Google Forms survey distributed to prospective students, teachers and administrators. The responses were cleaned, normalised and analysed to produce a prioritised set of functional and non-functional requirements using the MoSCoW technique. These requirements informed a formal system design phase in which the platform's structure was modelled using standard software-engineering artefacts - context diagrams, data-flow diagrams, use-case diagrams, sequence diagrams, class diagrams, deployment diagrams and an entity-relationship data model. The design was then realised through iterative implementation of the mobile application, the web dashboards and the backend API, with each sprint producing a testable increment of the system. The completed platform was deployed to accessible hosting environments and evaluated through functional testing across its core modules. Full details of each stage of this methodology are presented in Chapters Two, Three and Four.

5. Research Questions

- What connectivity, device and content-format constraints most significantly limit online learning for students in low-resource environments such as Cameroon?
- What combination of adaptive content delivery and offline-first data management best preserves learning continuity when network quality fluctuates or fails?

- How can a mobile e-learning architecture detect network conditions in real time and translate that information into an appropriate content-delivery decision without noticeable delay or disruption to the learner?
- To what extent does the resulting platform improve perceived learning continuity relative to the connectivity-related obstacles identified during the requirements phase?

6. Research Hypothesis

This project is primarily a design-and-implementation study rather than a strictly hypothesis-testing one; nevertheless, its design decisions rest on a testable working assumption: that a mobile e-learning platform which automatically classifies network quality into discrete tiers and adapts its content format accordingly will sustain measurably better learning continuity - fewer session abandonments and less time lost to buffering - than a platform offering only a single, fixed content format. This assumption is evaluated indirectly in Chapter Four through functional testing of the adaptive-delivery and offline-synchronisation modules.

7. Significance of the Study

This project is significant at several levels. For students in low-resource environments, it offers a practical means of maintaining access to educational content despite unreliable connectivity, directly addressing a barrier that, according to this project's own survey data, has caused nine in ten respondents to abandon an online lesson at least once. For teachers and institutions, it provides a lightweight platform for content delivery and progress monitoring that does not assume access to high-speed infrastructure. For the wider field of software engineering, it contributes a documented, reference architecture - spanning requirements engineering, system design, and full-stack implementation - for building connectivity-resilient, offline-first mobile applications, which may be adapted to other domains beyond education, such as mobile health and agricultural extension services, that face similar infrastructural constraints.

8. Scope of the Study

The scope of this project covers the complete software development life cycle of the Adaptis platform: requirements gathering and analysis; system and database design; implementation of a student-facing mobile application, a teacher and administrator web dashboard, and a backend API; and the deployment and functional evaluation of the resulting system. The mobile application targets the Android platform, and the adaptive delivery logic

covers three content tiers - high-definition video, standard-definition video, and an audio-and-text Lite Mode - selected according to measured network conditions.

9. Delimitation of the Study

This project is delimited to a single-institution, prototype-scale deployment intended for academic evaluation rather than public release. It does not extend to publication on the Google Play Store or Apple App Store, live video conferencing, artificial-intelligence-based content recommendation, or gamification features, all of which were explicitly classified as ‘Won’t Have’ items for this version during requirements prioritisation. Integration with a full third-party Learning Management System such as Moodle was evaluated but not adopted for this version; the platform instead uses a purpose-built Node.js and MongoDB backend. Multilingual (English/French) support is limited to a documented future enhancement rather than a delivered feature.

10. Definition of Keywords and Terms

Term	Definition
Adaptive content delivery	The automatic selection of a content format (video, audio or text) based on measured network conditions.
Quality of Experience (QoE)	The overall satisfaction and perceived performance experienced by an end user of a system, as distinct from raw technical performance metrics.
Quality of Service (QoS)	Objective, measurable network performance metrics such as bandwidth, latency and packet loss.
Offline-first architecture	A design approach in which local device storage, rather than a remote server, is treated as the primary source of truth, allowing an application to function without an active network connection.
Lite Mode	The lowest content tier in Adaptis, consisting of audio and text only, used when network conditions are poor or a connection is unavailable.
MoSCoW prioritisation	A requirements-prioritisation technique that classifies requirements as Must Have, Should Have, Could Have, or Won’t Have.

11. Organization of the Dissertation

This dissertation is organised into five chapters. Chapter One presents the general introduction, covering the background and context of the study, the problem statement, objectives, methodology, research questions, significance, scope and delimitation of the study. Chapter Two reviews the relevant literature, situating Adaptis within existing research on adaptive streaming, offline-first system design, and comparable e-learning platforms. Chapter Three presents the analysis and design of the system, including the requirements-engineering

process, the proposed methodology, and the architectural and behavioural models of the platform. Chapter Four describes the implementation of the platform and presents and evaluates the results obtained. Chapter Five concludes the dissertation with a summary of findings, the project's contribution to engineering and technology, recommendations, difficulties encountered, and directions for further work.

CHAPTER TWO

LITERATURE REVIEW

1. Introduction

This chapter establishes the theoretical and contextual foundation for Adaptis, an adaptive mobile e-learning platform designed to address connectivity constraints in low-resource environments such as Cameroon. It first defines the technical concepts on which the platform's design rests - the distinction between Quality of Service and Quality of Experience, adaptive bitrate streaming, offline-first system architecture, and the Model-View-ViewModel pattern - before reviewing the regional context and existing platforms that attempt to solve related problems. The chapter closes by identifying the specific gap in the literature that this project addresses.

2. General Concepts on Adaptive, Network-Aware E-Learning

2.1 Quality of Service versus Quality of Experience

Quality of Service (QoS) refers to objective, measurable network performance metrics, such as bandwidth, latency, jitter and packet loss (ITU-T, 2017). Quality of Experience (QoE), by contrast, refers to the overall satisfaction and perceived performance experienced by the end user of a system - a subjective measure that does not always correlate directly with QoS metrics. A video stream may satisfy a fixed QoS threshold and yet still produce a poor user experience if it buffers unpredictably or drops in quality at an inopportune moment. Adaptis is designed around the QoE principle: rather than optimising for a single technical metric, it uses live network conditions as a signal to preserve the learner's continuity of experience, even where this means deliberately serving a lower-quality format.

2.2 Adaptive Bitrate Streaming

Adaptive Bitrate (ABR) streaming is a technique, popularised by protocols such as HTTP Live Streaming (HLS) (Apple Inc., 2023), in which a media stream is encoded at multiple quality levels and the client switches between them based on measured network throughput. Adaptis extends this idea beyond a purely video-based bitrate ladder into a three-tier content model - high-definition video, standard-definition video, and an audio-and-text 'Lite Mode' - specifically tailored to bandwidth-constrained educational environments where even the lowest conventional video bitrate may be unusable.

2.3 Offline-First Architecture and Reactive UI Patterns

An offline-first design treats local device storage as the primary source of truth, synchronising with a remote server opportunistically rather than continuously (Google, 2023). This approach is well suited to environments where connectivity cannot be guaranteed. Adaptis pairs this storage strategy with the Model-View-ViewModel (MVVM) architectural pattern, which exposes data streams from the underlying model directly to the user interface through data-binding, so that UI components reflect state changes automatically as content is downloaded, cached or synchronised in the background.

2.4 Document-Oriented Data Storage

Adaptis further relies on a document-oriented NoSQL database (MongoDB) rather than a traditional relational database. This choice reflects the semi-structured nature of the platform's core data: a single lesson may simultaneously hold a video reference, an audio reference and a text transcript, representing the same educational content at different quality tiers, in addition to variable telemetry data describing network conditions at the time of access (MongoDB Inc., 2023).

3. Related Works

Literature on internet access in Sub-Saharan Africa and Cameroon consistently identifies infrastructure barriers - rather than a lack of motivation - as the primary obstacle facing learners (GSMA, 2023). A significant proportion of the population lacks consistent, affordable internet access, which makes mobile-first, data-conscious solutions a practical necessity rather than a design preference. Several existing platforms have attempted to respond to this reality, each with a different emphasis and, correspondingly, a different set of limitations.

Platform	Strength	Limitation
Kolibri	Excellent offline-content delivery for low-resource classrooms.	Lacks dynamic, real-time network adaptation; content must be pre-loaded rather than adjusted live.
Eneza Education	High accessibility via basic SMS and USSD channels.	Limited to text-only content; cannot deliver richer video or audio lessons.
uLesson	High-quality video lessons delivered via pre-loaded media.	Relies on bulk distribution rather than real-time network-aware adaptation.
Moodle	Mature, widely adopted Learning Management System with strong course-management tools.	Assumes largely stable connectivity for its core web and streaming workflows; offline support is limited and manual.

Taken together, these platforms show that the individual building blocks required for connectivity-resilient learning already exist in the literature and in practice - offline content packaging, lightweight text-based delivery, and pre-loaded high-quality media - but no single platform in the review combines them with live, QoE-based adaptation targeted specifically at education.

4. The Research Gap

While there is extensive research on QoE-driven adaptive streaming, it is almost exclusively focused on entertainment video (Laghari & Connelly, 2012). There is a clear gap for an education-specific application that integrates live, real-time QoE-based adaptation; offline-first resilience; and a mobile-first, multi-tier delivery system, within a single, cohesive platform. Adaptis is designed to fill this gap by merging these principles into a solution built explicitly for the infrastructure realities of Cameroon and comparable low-resource environments.

5. Partial Conclusion

This chapter has situated Adaptis within the technical concepts of QoE-driven adaptation, adaptive bitrate streaming, offline-first architecture and document-oriented data storage, and has reviewed existing e-learning platforms that address related but narrower problems. The review confirms that no existing platform combines real-time network-aware adaptation with offline-first resilience in a mobile-first educational product, which motivates the analysis, design and implementation presented in the following two chapters.

CHAPTER THREE

ANALYSIS AND DESIGN

1. Introduction

This chapter presents the personal work carried out by the project team in analysing the research problem and designing a solution. It begins by describing the methodology used to gather, review and prioritise system requirements from prospective users. It then presents the design of the platform, covering the visual/UX design system and the conceptual database design. The chapter continues with the global architecture of the solution and closes with a description of the resolution process, expressed through the platform's context, data-flow, use-case, sequence, class and deployment diagrams.

2. Proposed Methodology

2.1 Requirements Engineering Process

Requirements for Adaptis were gathered through an iterative, seven-stage lifecycle beginning with stakeholder identification, followed by field data collection, requirement classification, review for completeness and clarity, prioritisation, stakeholder validation, and continuous refinement across successive sprints. Stakeholders were grouped into direct operational actors and structural environment constraints, summarised in Table 1.

Table 1: Stakeholder Analysis Summary

Stakeholder	Key Interest	Influence on Requirements	Priority
Students (primary users)	Uninterrupted access to learning material despite poor connectivity and limited data.	Functional requirements for Lite Mode and offline downloads.	High
Teachers (primary users)	Effective content delivery and student tracking.	Requirements for simple content upload and progress-tracking tools.	Medium
Internet Service Providers	Bandwidth management across their networks.	Technical constraints shaping the network-aware adaptation algorithm.	High
University administrators	Student engagement and retention metrics.	Requirements for reporting and analytics.	Low

2.2 Field Data Collection

The survey method was selected as the primary requirements-gathering technique. A structured, twenty-one-question instrument combining multiple-choice, checkbox, five-point Likert-scale and open-ended items was designed and deployed using Google Forms, chosen for its accessibility on mobile hardware, low registration friction, respondent anonymity, and structured, analysable output. The survey collected fifty-nine (59) unique responses. Raw responses were cleaned prior to analysis: platform-name entries were case-standardised, informal mentions of general-purpose tools (e.g. WhatsApp) were retained but categorised separately from dedicated e-learning platforms, French-language responses were translated and mapped to equivalent English categories, and optional fields left blank were preserved as null values rather than assigned an assumed value.

2.3 Quantitative Findings

Table 2: Key Survey Findings and their Influence on Requirements

Finding	Value	Requirement Influenced
Respondents who are students	95%	System is primarily designed around the student user role.
Respondents relying on a smartphone (exclusively or combined with a laptop)	83%	Mobile-first design is mandatory.
Respondents rating their connectivity at or below average	95%	Adaptive delivery is the core, non-negotiable feature.
Respondents who face network problems always or sometimes	97%	Reliability and graceful degradation requirements.
Respondents who have abandoned learning due to poor internet	90%	Confirms the core problem the system must solve.
Respondents selecting offline download as a desired feature	86%	Offline access prioritised as a Must Have.
Respondents wanting automatic progress saving	92%	Automatic local save and background sync required.

These findings, together with device-ownership data showing that 44% of respondents own a smartphone only and a further 39% own both a smartphone and a laptop, confirmed that the platform had to be mobile-first, data-conscious, and resilient to frequent connectivity loss by design rather than as an afterthought.

2.4 Requirement Classification and Prioritisation

Fifty requirements were identified in total and classified as either Functional (33 requirements, describing what the system does) or Non-Functional (17 requirements, describing how well the system performs). Each requirement was reviewed against four criteria - completeness, clarity,

technical feasibility and dependency relationships - and prioritised using the MoSCoW technique, which weighs survey evidence of user importance against the team's technical feasibility assessment.

Table 3: MoSCoW Prioritisation Criteria

Priority	Criteria	Representative Requirements
Must Have	Directly solves the core problem; survey evidence exceeds 80% of respondents; technically feasible for the team.	Adaptive streaming, offline download, auto-save, secure login.
Should Have	Significantly improves user experience; moderate survey evidence; feasible with additional time.	Manual data-saver toggle, quiz module, notifications, progress dashboard.
Could Have	Adds value but is not essential; technically possible but may exceed the project timeline.	AI-based content recommendations, gamification, advanced analytics.
Won't Have (v1)	Conflicts with project constraints or exceeds available resources for this version.	Live video conferencing, third-party payment integration.

The Must Have requirements form the core of the platform delivered and evaluated in this dissertation: user authentication, adaptive three-tier content delivery, offline access with automatic synchronisation, and a mobile-responsive, low-literacy-friendly interface. Should Have items - the quiz module, notifications, and teacher-student messaging - were implemented as secondary features once the core adaptive pipeline was stable. Could Have and Won't Have items were consciously deferred, as documented in the delimitation of the study in Chapter One.

2.5 Requirement Validation

Requirements were validated with stakeholders through three complementary methods: clickable prototypes and demonstrations reviewed with the course supervisor and sample users; a structured stakeholder questionnaire; and a set of representative user stories - for example, 'As a student with poor mobile data, I want the app to automatically switch to audio-only mode so I can continue listening to my lecture without the video buffering' - used to confirm that each Must Have requirement mapped to a concrete, recognisable user need before implementation began.

2.6 Risk Assessment and Reluctance Mitigation

Because target users have previously been let down by unreliable digital tools, several categories of user reluctance were anticipated and addressed directly in the design, converting each concern into a concrete software guarantee.

Table 4: Reluctance Mitigation Summary

Area of Reluctance	Underlying Barrier	Design Response
Cost	Fear that adaptive streaming could deplete a limited data bundle unexpectedly.	A Data Transparency module forecasts megabyte usage per lesson; users may lock the app into a permanent Low-Data Mode.
Continuity	Fear of losing progress when a session is interrupted (90% had abandoned a lesson previously).	Local-first caching with a visible ‘Live-Save’ indicator confirms progress is preserved every few seconds and synced once connectivity returns.
Device performance	Concern that a heavy application will slow down or drain low-end devices.	A lean build footprint and server-side computation of adaptive transitions minimise local CPU and battery load.
Control	Dislike of an app that locks users into a mode they did not choose.	A manual override is always available alongside any automatic content-quality change.

3. Design

3.1 Application Identity

A unified visual brand identity was created to ensure that the platform presents a consistent, professional appearance whether accessed by a student on a low-end smartphone or by a teacher through a desktop web browser. The logo uses a simple, modern mark built around a stylised letter ‘A’ whose base is shaped like an upward arc, symbolising forward progress and adaptation, paired with the tagline ‘Learn. Adapt. Grow.’ Three launcher-icon variants - a solid-colour standard tile, a light variant, and a high-contrast dark variant for devices in dark mode - were produced to ensure the icon remains legible across device settings.



Figure 1: Adaptis application logo and launcher-icon variants

3.2 Visual and Interaction Design

The interface was prototyped collaboratively in Figma before implementation began, allowing the team to test navigation flows and refine layouts across all screens prior to writing frontend code. The design prioritises accessibility and simplicity so that the application does not overwhelm learners with limited digital literacy. A restrained colour palette of green, white and deep grey/black was adopted: green is reserved strictly for actionable elements such as buttons and status indicators; white forms clean, distraction-free backgrounds; and black/deep grey is used for headings and body text to maximise readability without causing eye strain during extended reading. The layout combines a fixed top navigation bar with a collapsible side menu, and organises related information into simple, border-free cards, avoiding heavy decorative artwork so that pages remain lightweight and responsive.

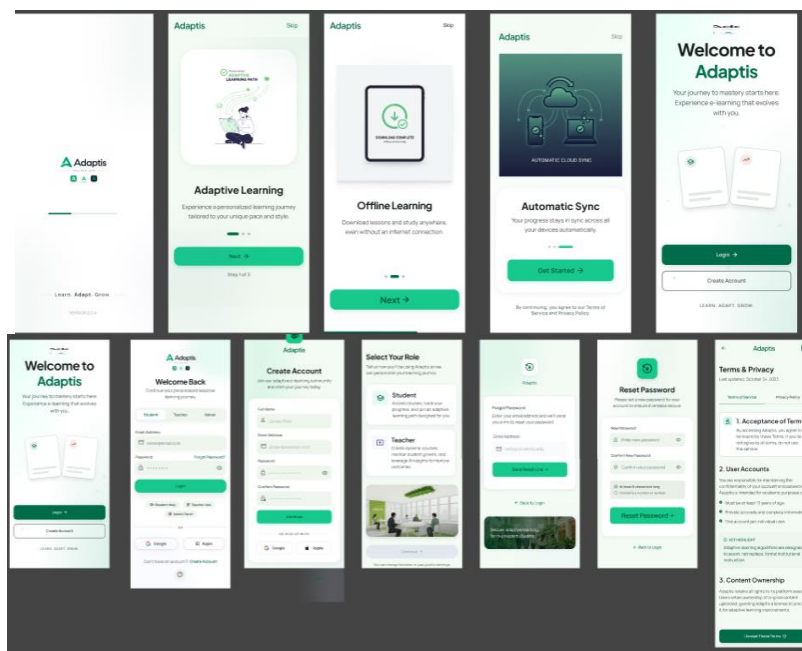


Figure 2: Figma prototype screens (1 of 2) used to validate the visual and interaction design

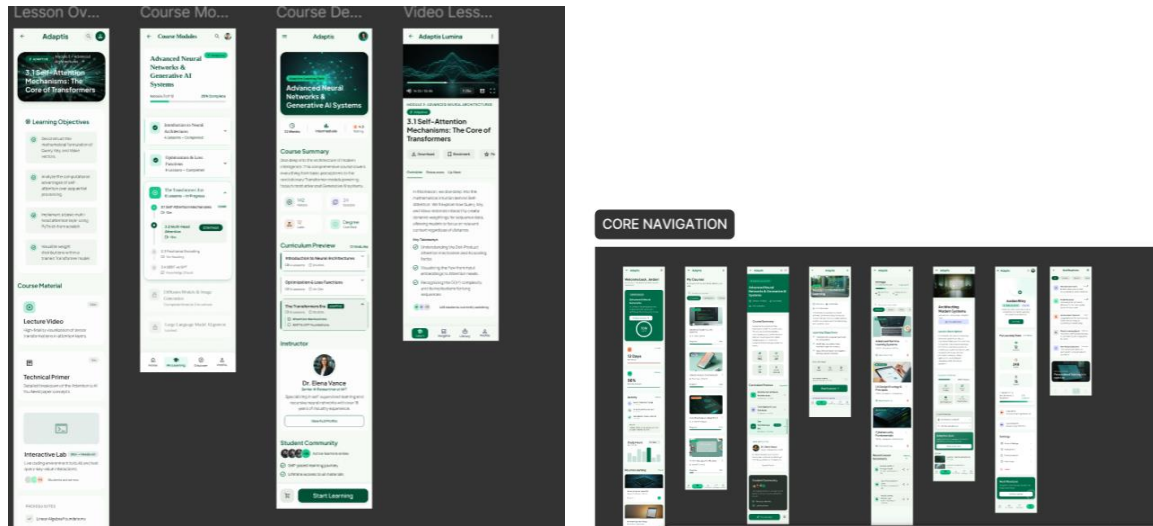


Figure 3: Figma prototype screens (2 of 2) used to validate the visual and interaction design

3.3 Conceptual Database Design

The conceptual database design defines the system's entities, attributes, relationships and business rules that underpin the physical MongoDB schema described in Chapter Four. Six core entities were identified: USER, COURSE, LESSON, ENROLLMENT, MESSAGE and NETWORK_LOG.

Table 5: Core Database Entities

Entity	Key Attributes	Purpose
USER	User_ID (PK), Full_Name, Email (unique), Password_Hash, Role, Location, Device_Type, Created_At	Stores profile information for students, teachers and administrators.
COURSE	Course_ID (PK), Course_Title, Description, Category, Difficulty_Level, Created_By (FK → USER), Created_At	Represents a learning programme created by a teacher.
LESSON	Lesson_ID (PK), Course_ID (FK), Lesson_Title, Lesson_Content_Text, Video_URL_HD, Video_URL_SD, Audio_Text_URL, Lesson_Order, Duration_Minutes	Stores learning content together with its three adaptive media variants.
ENROLLMENT	Enrollment_ID (PK), User_ID (FK), Course_ID (FK), Enrollment_Date, Progress_Percentage, Status	Associative entity resolving the many-to-many relationship between USER and COURSE.
MESSAGE	Message_ID (PK), Sender_ID (FK), Receiver_ID (FK), Message_Content, Timestamp, Sync_Status	Records communication between users, including messages composed while offline.

Entity	Key Attributes	Purpose
NETWORK_LOG	Log_ID (PK), User_ID (FK), Network_Condition, QoE_Score, Latency_ms, Timestamp	Stores network-performance telemetry used to drive adaptive decisions.

The principal business rules enforced by this schema are that every course must be created by exactly one user holding the Teacher role; that a lesson cannot exist outside a parent course, and is removed if its parent course is deleted; that a user may enrol in many courses and a course may hold many enrolled users, resolved through the ENROLLMENT entity; that each message carries exactly one sender and one receiver, with a Sync_Status field that allows messages composed offline to be queued and delivered once connectivity is restored; and that every lesson must expose at least one of its three adaptive content formats at all times.

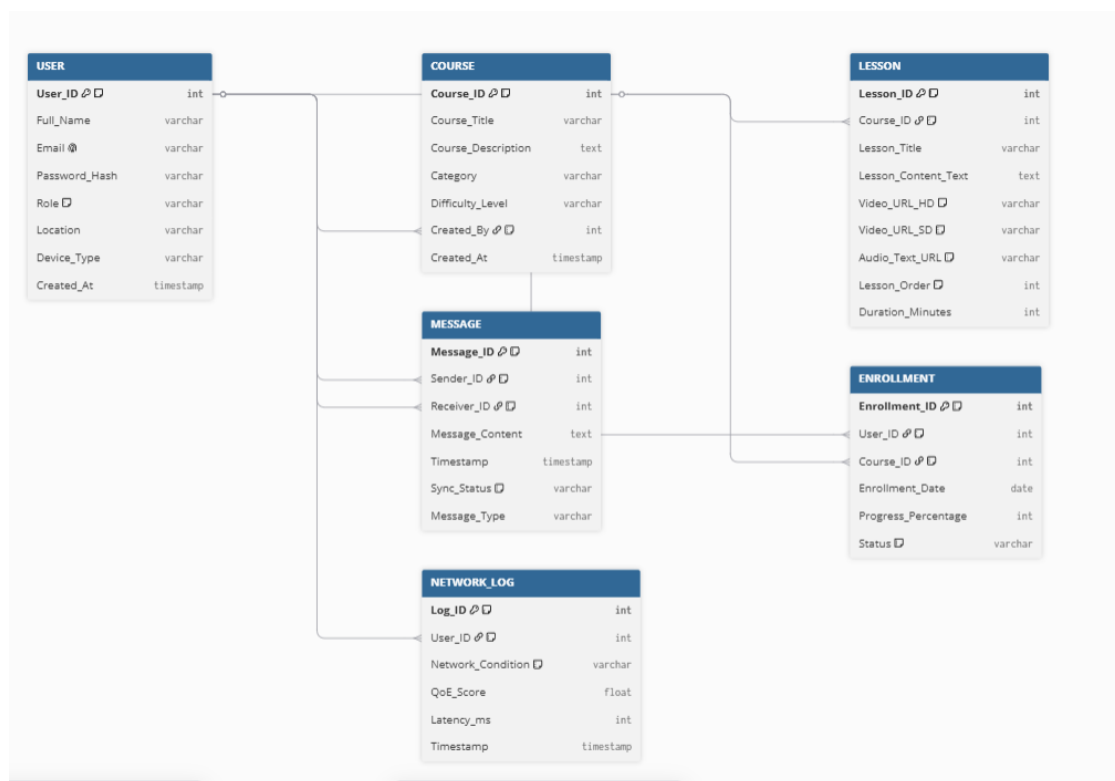


Figure 4: Entity-Relationship Diagram of the Adaptis database

4. Global Architecture of the Solution

Adaptis follows a client-server architecture composed of three principal tiers: a Flutter mobile client used by students, a React web dashboard used by teachers and administrators, and a Node.js/Express backend backed by a MongoDB Atlas database. The technology choices behind each layer are summarised in Table 6.

Table 6: Technology Stack Summary

Layer	Technology	Purpose
Mobile frontend	Flutter (Dart)	Cross-platform mobile application with native performance on Android.
Web frontend	React with Vite	Single-page dashboard for teachers and administrators.
Connectivity sensing	connectivity_plus package	Detects connection type and measures real-time network quality.
Local storage	Device-local key-value/document store	Offline data storage and caching of downloaded lessons.
Backend API	Node.js and Express.js	RESTful API handling authentication, business logic and data access.
Database	MongoDB Atlas (with Mongoose)	Cloud-hosted document database storing users, courses, lessons and telemetry.
Authentication	JSON Web Tokens (JWT) and bcrypt	Stateless authentication and secure password storage.
Charts and analytics	Recharts	Visualises platform usage and network-condition metrics on the admin dashboard.

The deployment diagram in Figure 8 (Section 5.6) illustrates how these components are physically distributed across student devices, browser clients, and cloud infrastructure, and how they communicate over HTTPS using REST APIs and adaptive HTTP-based media delivery.

5. Description of the Resolution Process

This section describes, through a series of standard software-engineering diagrams, how Adaptis resolves the research problem stated in Chapter One - namely, how the system senses network conditions and internal processes cooperate to keep a learner continuously supplied with an appropriate form of content.

5.1 Context Diagram

The context diagram defines the boundary of the system and the external entities with which it exchanges data, without exposing internal processing. Adaptis interacts with three external entities: the Instructor, who supplies authentication credentials, course materials and assessments and receives upload and management feedback; the Student, who supplies login credentials, course requests and submissions and receives adaptive content, notifications and results; and the Network Environment, which supplies bandwidth, latency and packet-loss metrics that the system continuously monitors, without returning data directly to it.

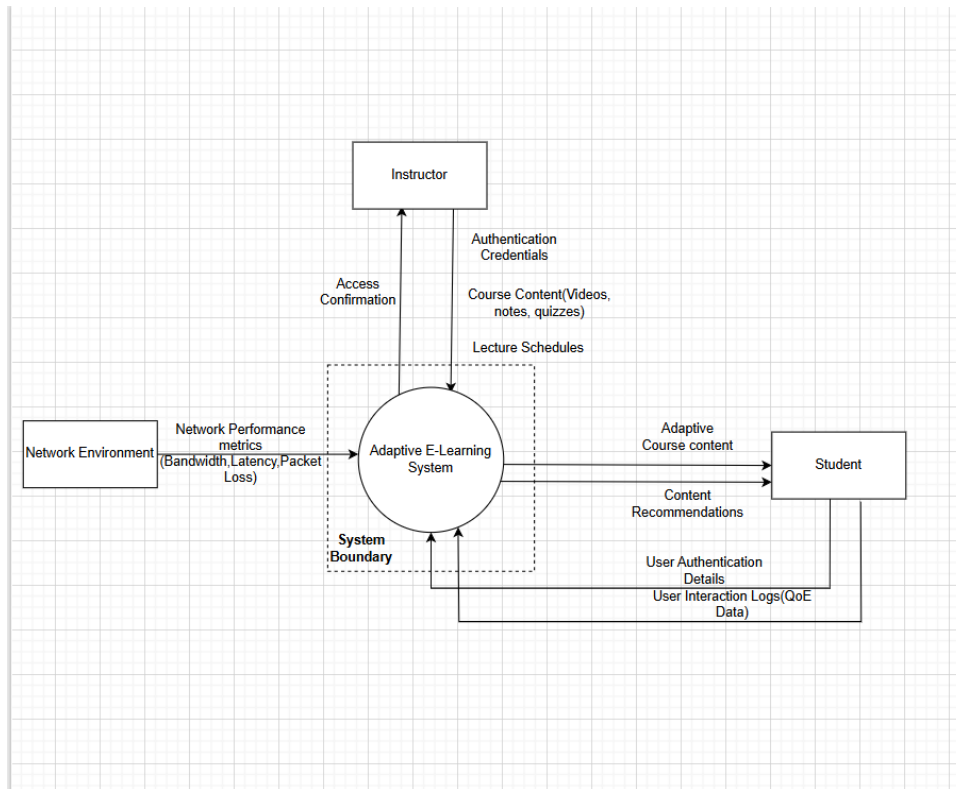


Figure 5: Context Diagram of the Adaptis platform

5.2 Data Flow Diagram

The Level 1 Data Flow Diagram (DFD) exposes the internal processes and data stores that the context diagram abstracts away. Three major processes were identified: Authenticate User, which verifies credentials and issues secure session tokens; Monitor Network Speed, which continuously measures bandwidth, latency and availability and produces a network-quality classification; and Manage Course Content, which retrieves and adapts course materials according to the network-quality classification produced by the monitoring process, and streams the resulting content to the student. Local device storage persists offline content, cached progress and quiz attempts, synchronising them back to the server once connectivity is restored. The diagram was validated against standard systems-analysis checks: balanced inputs and outputs, correct process transformation logic, restricted direct access to data stores, consistent naming, and a clear separation between external entities and internal processes.

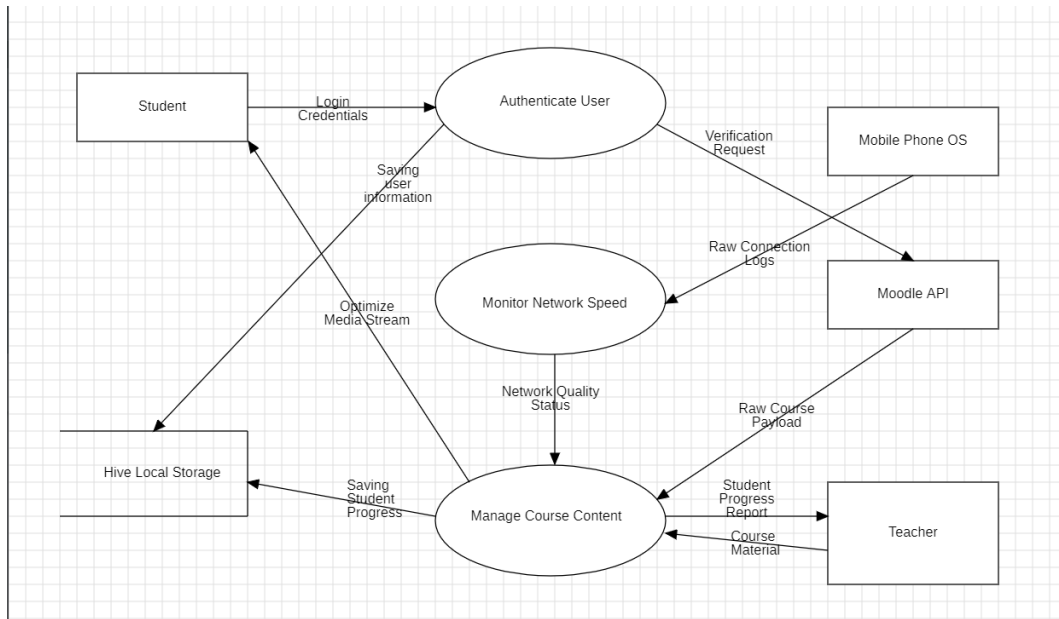


Figure 6: Level 1 Data Flow Diagram of the Adaptis platform

5.3 Use Case Diagram

The use case diagram models the interactions between the system's four actors - Student, Instructor, Administrator, and an Automated Monitoring System - and the services available to each. The Student registers and logs in, browses courses, downloads content for offline learning, submits assignments, takes quizzes, receives notifications, and enables data-saver mode. The Instructor uploads materials, creates quizzes and assignments, and monitors student progress. The Administrator manages accounts and system configuration. The Automated Monitoring System, running continuously in the background, monitors connection quality, adjusts streaming quality, and synchronises offline data. Two include relationships were modelled explicitly: Stream Content includes Monitor Connection and Speed, and Synchronise Progress includes Monitor Connection and Speed, ensuring that adaptive operations always act on current network information.

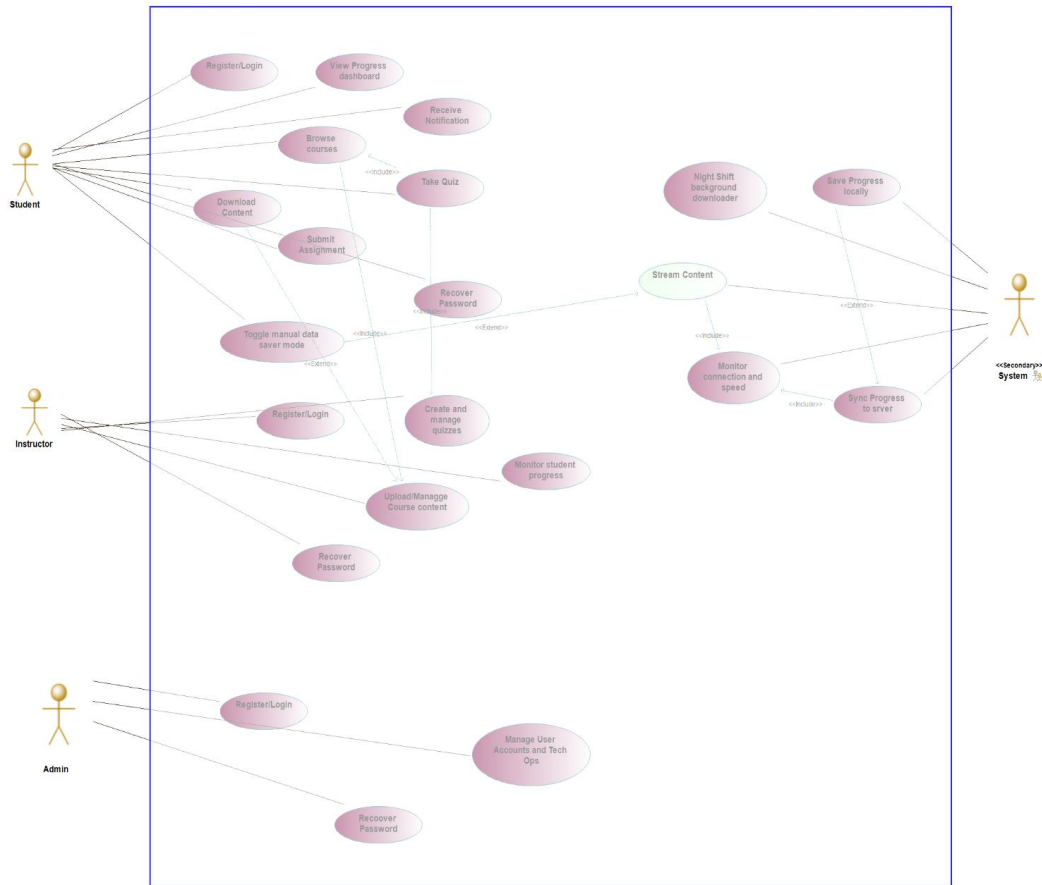


Figure 7: Use Case Diagram of the Adaptis platform

5.4 Sequence Diagram

The sequence diagram traces the chronological interaction between the Student, Instructor, Administrator, the Automated Monitoring System and the Core System across a typical session: authentication and secure session initialisation; content access, in which instructors upload material that students subsequently browse; offline download and local storage of learning materials; assignment submission and quiz completion with instructor feedback; adaptive streaming, in which the monitoring module evaluates network conditions and switches between HD video, SD video, audio-only and text delivery as needed; synchronisation of locally stored progress once connectivity is restored; administrative operations; and background optimisation, in which downloads are scheduled during off-peak network periods.

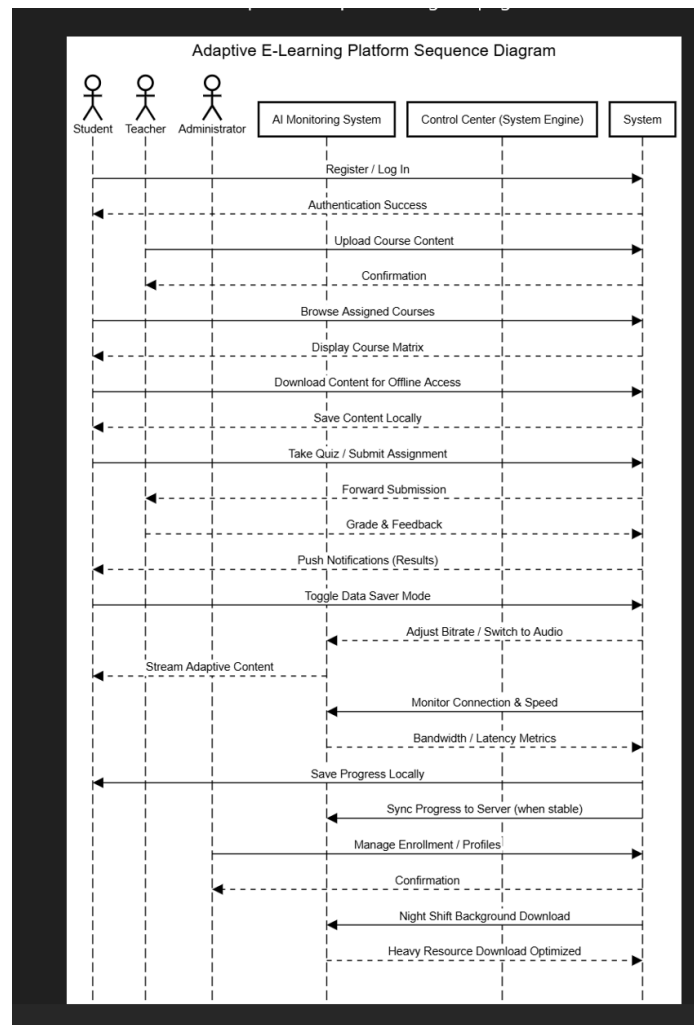


Figure 8: Sequence Diagram of a typical Adaptis session

5.5 Class Diagram

The class diagram defines the object-oriented structure underlying the implementation. A base User class provides shared attributes (user ID, name, email, password) and methods (login, logout), specialised by Student, Instructor and Administrator subclasses through inheritance. Course and Quiz classes model the platform's core learning content, while four supporting technical modules - the Connectivity module, Media Player, Offline Storage, and Notification module - encapsulate, respectively, network-quality sensing, adaptive playback, local caching and synchronisation, and alerting. Association relationships connect users to the courses, quizzes and notifications they interact with, while dependency relationships capture temporary interactions between technical modules, such as the Media Player consulting the Connectivity module before selecting a content tier.

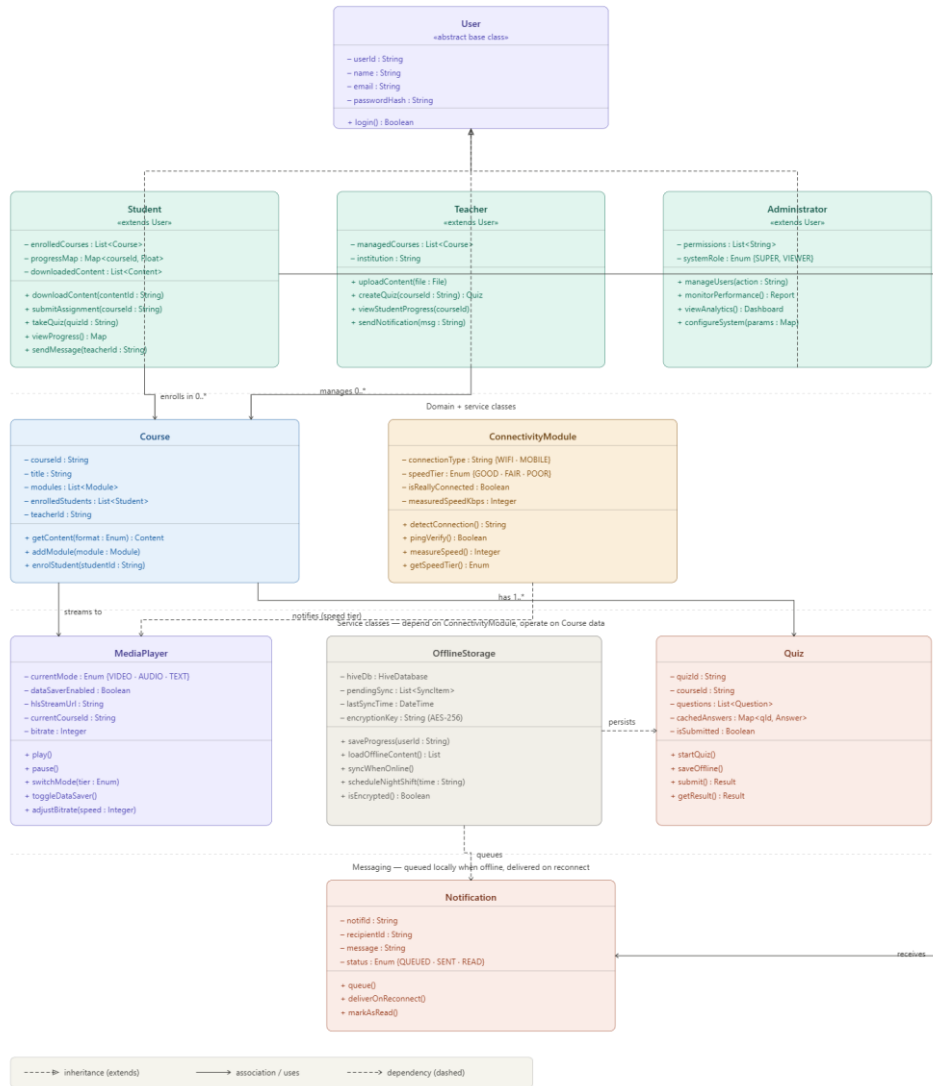


Figure 9: Class Diagram of the Adaptis platform

5.6 Deployment Diagram

The deployment diagram describes the physical distribution of the system across hardware and cloud infrastructure: the Flutter mobile application, running on student and instructor Android devices with encrypted local storage; a web browser interface used by instructors and administrators for course and platform management; a Node.js cloud application server handling authentication, synchronisation and adaptive media logic; the MongoDB Atlas database; and a content delivery network that reduces media latency for users across the region. Mobile and web clients send requests to the application server over HTTPS using REST APIs; media is streamed through the content delivery network; and offline data is synchronised automatically once connectivity becomes available.

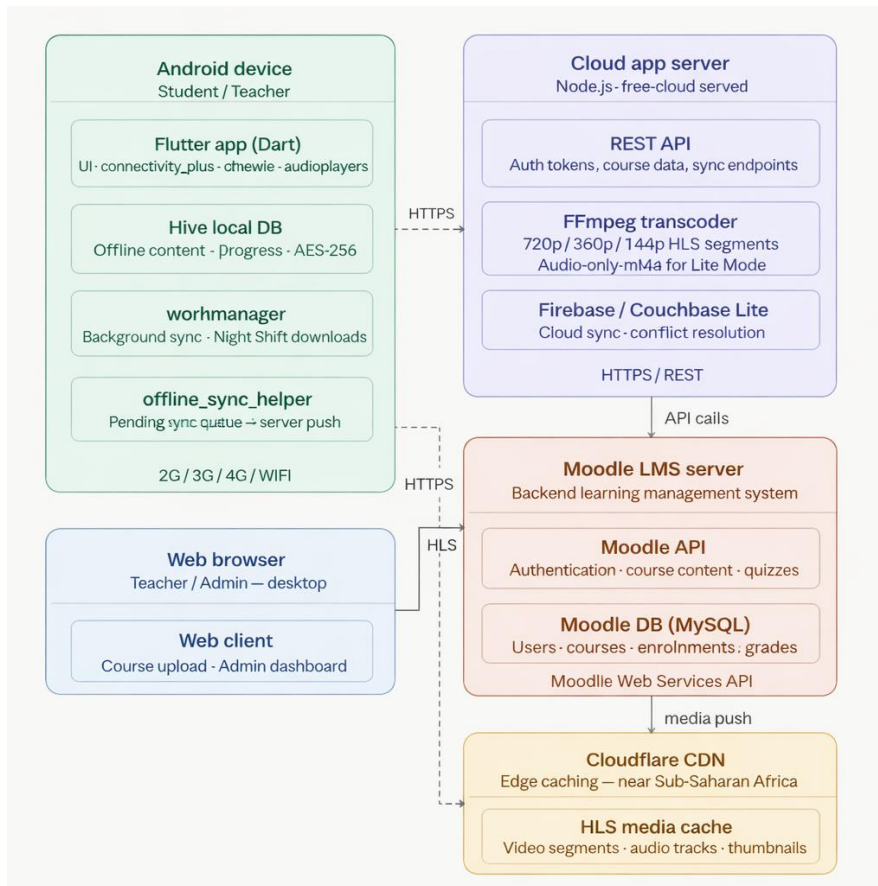


Figure 10: Deployment Diagram of the Adaptis platform

6. Partial Conclusion

This chapter has presented the analysis and design work underpinning Adaptis: a survey-driven requirements process that identified and prioritised fifty requirements; a visual and database design aligned with the needs of low-literacy, low-bandwidth users; a three-tier client-server architecture; and a complete behavioural and structural model of the system expressed through six standard software-engineering diagrams. Chapter Four now turns to how this design was implemented and the results obtained.

CHAPTER FOUR

IMPLEMENTATION AND RESULTS

1. Introduction

This chapter describes how the design presented in Chapter Three was translated into a working system, and presents the results obtained from that implementation. It covers the tools and materials used, the implementation process for the mobile application, web dashboards and backend API, the results obtained after deployment, and an evaluation of the solution against the objectives stated in Chapter One.

2. Tools and Materials Used

The mobile application was built using the Flutter SDK and the Dart programming language, chosen for its ability to compile a single codebase to smooth, natively performing Android builds even on low-end and mid-range hardware. The web dashboards for teachers and administrators were built using React with Vite as the build tool, styled with custom CSS and Lucide React icons, and visualised using the Recharts charting library. The backend was built using Node.js and the Express.js framework, with Mongoose used to define schemas over a MongoDB Atlas cloud database. Authentication was implemented using JSON Web Tokens (JWT) and the bcrypt hashing algorithm. Figma was used throughout for interface prototyping, and the completed mobile and web applications were deployed using Expo and Replit Deployments respectively.

3. Description of the Implementation Process

3.1 Student Mobile Application

The student-facing mobile application was implemented as a complete walkthrough of the platform's core screens: a splash screen and onboarding sequence, login and registration forms, the student home dashboard, course catalogues, lesson lists, a media playback screen, quiz pages, an offline download library, and a progress-tracking screen. To demonstrate the adaptive-delivery feature concretely during development and evaluation, a network-quality simulation controller was built into the application, allowing the current connection tier to be switched between Good (automatic HD video playback), Fair (automatic SD video playback), and Poor (automatic fallback to an audio track with text summaries), in addition to a manual Data Saver switch that a student may enable at any time to force the lightweight format regardless of measured network quality. Lesson and progress data are cached locally using the device's local storage, laying the groundwork for full offline lesson downloads and offline quiz

submission. Standard Material Design icons are used throughout so that navigation remains familiar to users regardless of technical background.

3.2 Teacher and Administrator Web Dashboard

The web dashboard was implemented as a single-page application using React and Vite. The teacher portal provides tools for creating and managing courses, arranging lesson lists, designing multiple-choice quizzes and reviewing student grades; its lesson-creation screen illustrates how an uploaded video is split automatically into the platform's three adaptive formats (720p, 360p, and an audio/text variant). The administrator portal provides a platform-wide view for reviewing user accounts, monitoring active courses, and auditing system activity, including charts summarising how many students are currently operating in Lite Mode, buffering frequency, and the distribution of connections across Cameroonian cities such as Yaoundé, Douala, Buea and Bamenda, to keep the evaluation grounded in the local network context the platform targets.

3.3 Backend API and Database

The backend exposes a RESTful API over HTTPS, organised into logical route groups - authentication, courses, lessons, assignments, quizzes, progress, messages and notifications - each delegating to a controller and an underlying service layer responsible for business logic and database access. All requests and responses use JSON, with consistent HTTP status codes (200/201 for success; 400, 401, 403, 404 and 500 for the corresponding error conditions), which keeps error handling in the Flutter client predictable. Twelve Mongoose models were defined, corresponding to the entities identified during design: User, Course, Module, Lesson, Assignment, Submission, Quiz, Question, Progress, Message, Notification and ActivityLog. The Lesson model is central to the adaptive feature: every lesson document stores a videoUrl, an audioUrl and a textContent field side by side, so that a single API call returns all three representations of a lesson together. Indexes were applied to the fields most frequently used in query conditions - a unique index on User.email, and indexes on the courseId and userId fields of the Lesson, Assignment and Progress models - to keep queries fast as data volume grows.

3.4 Authentication and the Adaptive Learning Engine

Authentication uses JSON Web Tokens: on successful login, the server issues a signed token containing the user's ID and role, valid for thirty days, which the Flutter client attaches to every subsequent request. A dedicated middleware verifies this token on protected routes and rejects missing, expired or tampered tokens with a 401 response before the request reaches its handler. Passwords are hashed with bcrypt before storage and are never stored or transmitted in plain

text. A second, role-based authorisation middleware restricts specific routes to specific roles - for instance, a Student calling the course-creation endpoint receives a 403 response regardless of token validity - enforcing the platform's Student/Teacher/Administrator permission boundaries at the API level rather than only in the interface.

The adaptive learning engine itself is implemented as a deliberate division of responsibility between backend and client: the backend's role is limited to guaranteeing that all three content formats for a lesson are always present, consistently structured, and returned together in a single response; the decision about which format to display is made entirely on the Flutter client, which evaluates the device's network quality in real time using the `connectivity_plus` package. This design allows the application to switch formats instantly when network quality changes mid-session, without a round trip to the server, and it means the same architecture continues to work when a student is fully offline: once a lesson - including all three of its content variants - has been downloaded to local storage, adaptive switching keeps functioning using only locally cached data.

3.5 Deployment

For evaluation purposes, the student mobile application was deployed using Expo, which allows the application to be installed and run on Android devices - via Expo Go, a QR code, or a direct deployment link - without requiring publication to the Google Play Store. Both the Admin Portal and Teacher Portal were deployed using Replit Deployments, giving administrators and teachers secure browser-based access to the platform. The backend server loads its configuration from environment variables at startup - including the MongoDB connection string, JWT secret and server port - stored in a `.env` file excluded from version control, and only begins accepting requests once its database connection is confirmed.

4. Presentation and Interpretation of Results

4.1 Mobile Application

The mobile application was made available to reviewers through the deployment link and QR code shown in Figure 9. Figures 10 through 12 present the login screen, the learning interface, and the lesson-delivery screens produced by the implementation described above.



Figure 11: Adaptis mobile application QR code and deployment access point

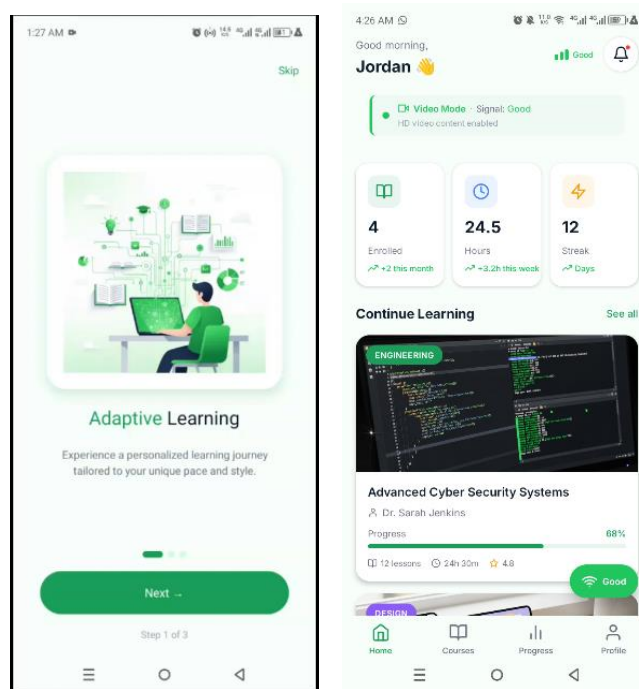


Figure 12: Mobile application login screens

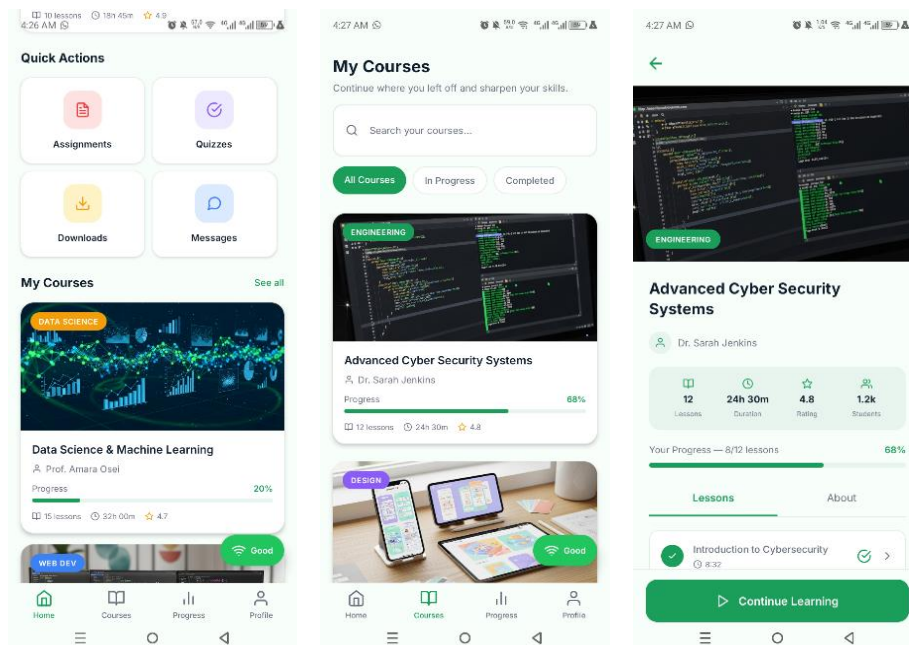


Figure 13: Student mobile learning interface

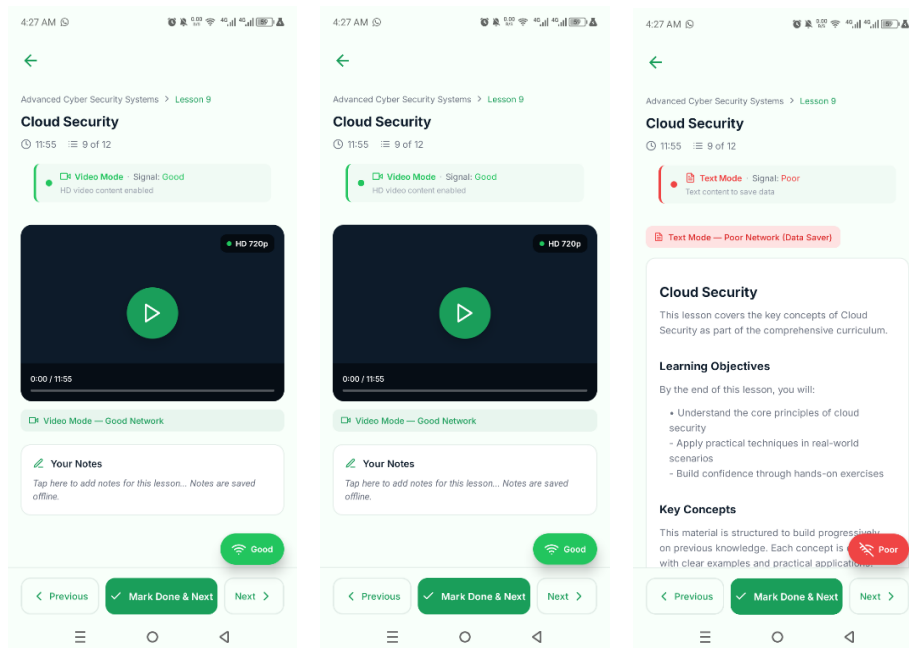


Figure 14: Adaptive lesson delivery across content tiers (1 of 2)

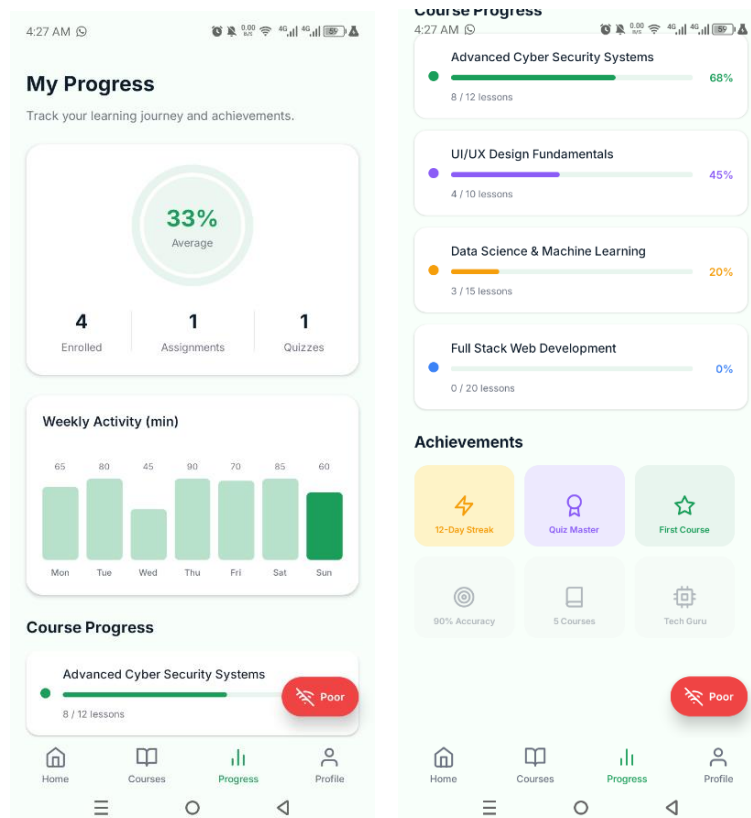


Figure 15: Adaptive lesson delivery across content tiers (2 of 2)

4.2 Administrator and Teacher Web Portals

The Admin Portal, deployed and reachable through Replit, provides the platform-wide management views shown in Figure 15, while the Teacher Portal, shown in Figure 16, provides the course, lesson and grading tools described in Section 3.2.

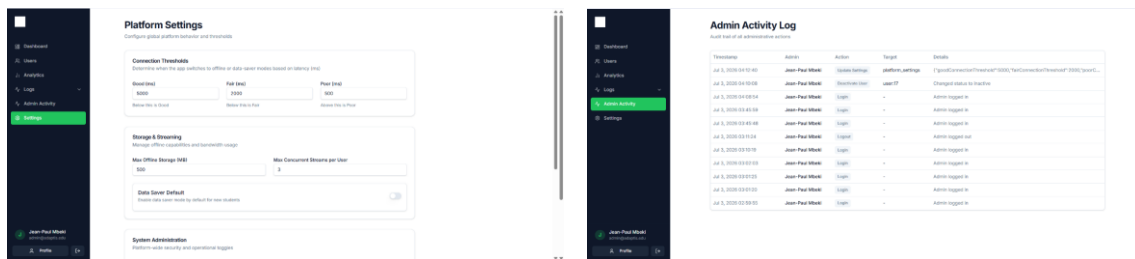


Figure 16: Admin Portal dashboard screens (1 of 2)

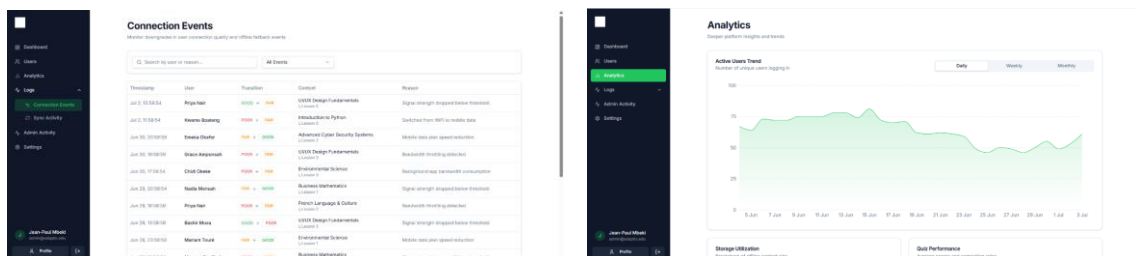


Figure 17: Admin Portal dashboard screens (2 of 2)

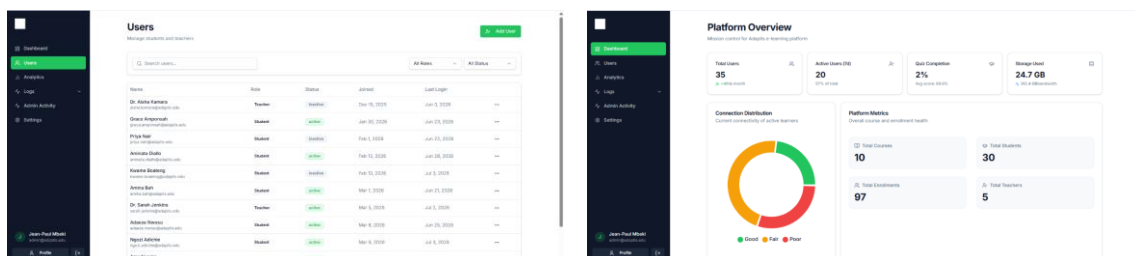


Figure 18: Admin Portal analytics and user-management screens

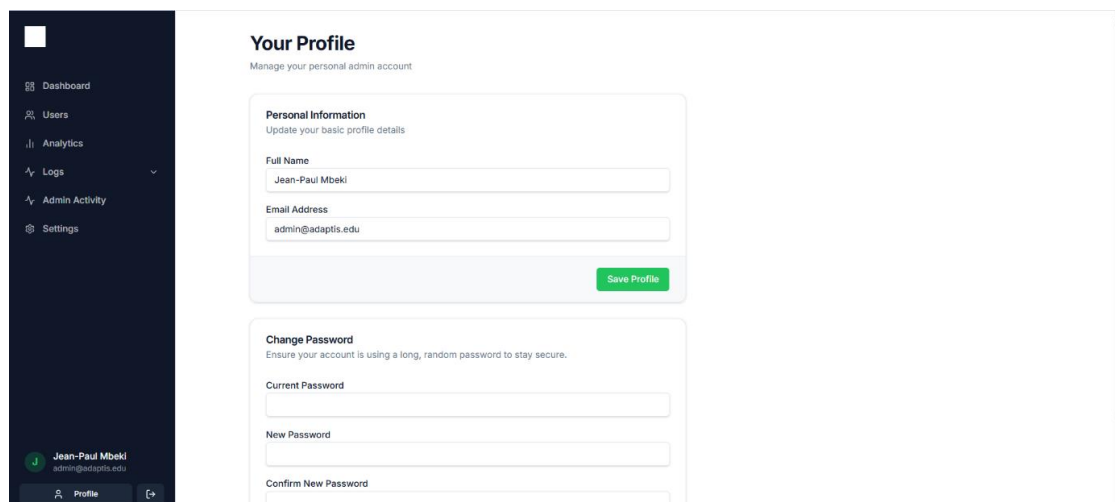


Figure 19: Admin Portal system-monitoring screen

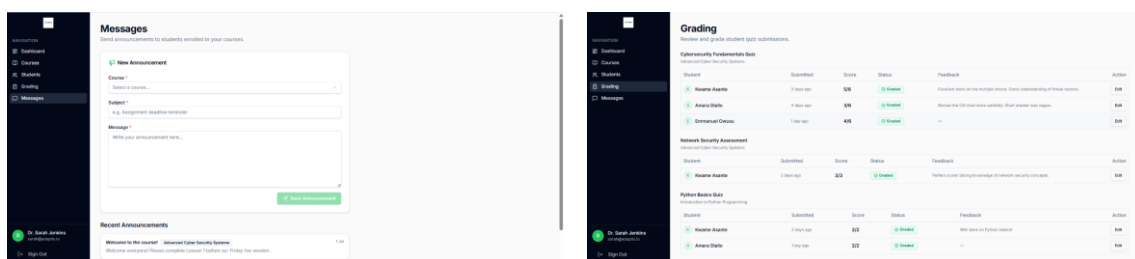


Figure 20: Teacher Portal course-management screens

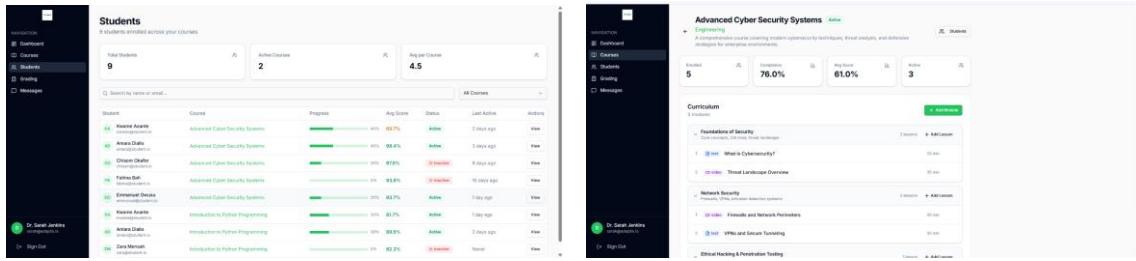


Figure 21: Teacher Portal lesson and quiz-creation screens

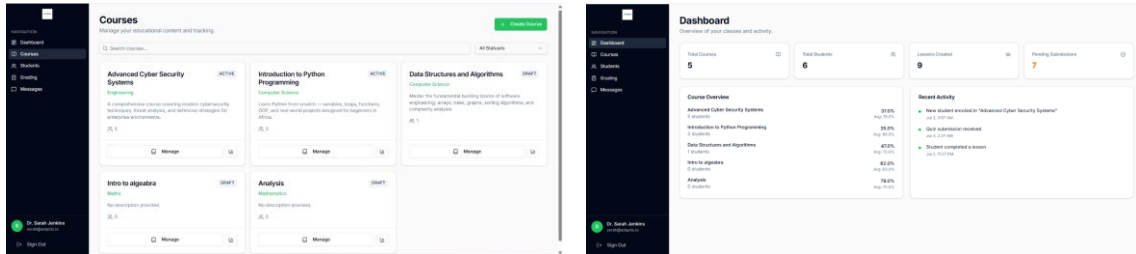


Figure 22: Teacher Portal student-grading and messaging screens

4.3 Testing

Following deployment, the platform was tested across its core modules: user registration; authentication for students, teachers and administrators; navigation across all modules; course access; dashboard functionality; general responsiveness; and cross-platform accessibility between the mobile application and the two web portals. A small number of runtime issues were identified and documented during this process, illustrated in Figure 20, and were addressed through configuration and environment-variable corrections between the development and production environments.

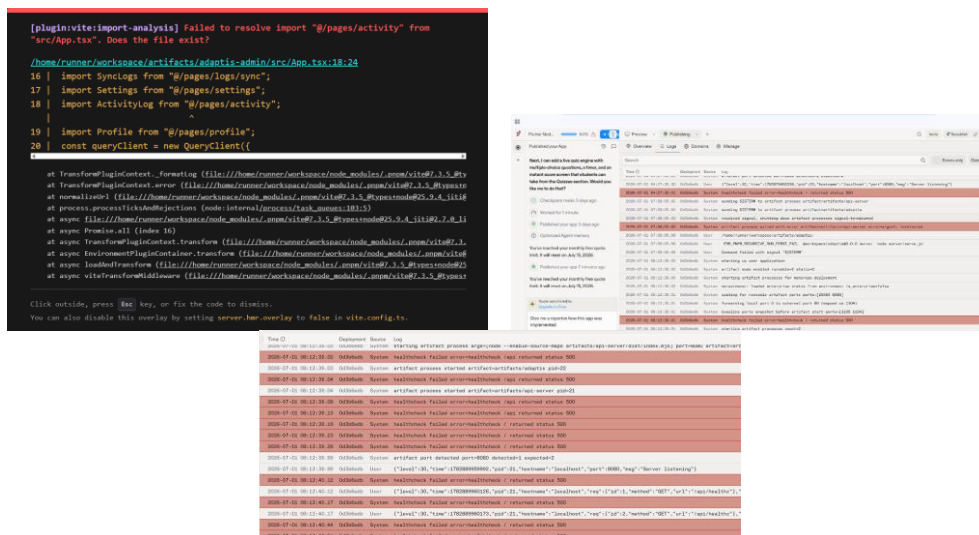


Figure 23: Runtime issues observed and resolved during post-deployment testing

5. Evaluation of the Solution

Measured against the objectives set out in Chapter 10, the delivered platform satisfies its core requirements. The three-tier adaptive delivery pipeline - HD video, SD video, and an audio-

and-text Lite Mode - functions end to end, with the client selecting a format based on measured network quality and switching between formats without requiring a server round trip, directly answering the reliability and continuity concerns identified in the requirements survey. Role-based authentication and authorisation correctly restrict Student, Teacher and Administrator capabilities at the API level. Course, lesson, assignment, quiz, progress, messaging and notification data are all modelled and served through a working REST API backed by MongoDB Atlas, and both web portals and the mobile application were successfully deployed and reached by reviewers using the credentials and links documented above.

Relative to comparable platforms reviewed in Chapter Two, Adaptis's principal contribution is the combination, in a single mobile-first product, of live QoE-based tier switching (which Kolibri and uLesson do not provide), full multimedia lesson content rather than text alone (which Eneza Education does not provide), and an offline-first data model with automatic synchronisation once connectivity returns. The evaluation also surfaced genuine limitations, discussed further in Chapter Five: the mobile application is currently distributed through Expo rather than an app-store listing, some Should Have features such as real-time push notifications remain partially implemented, and formal performance testing under simulated 2G/3G network conditions - as opposed to the manual network-tier simulation used during development - had not yet been carried out at the time of writing.

6. Partial Conclusion

This chapter has described the implementation of Adaptis across its mobile, web and backend components, presented the results obtained from its deployment, and evaluated those results against the project's stated objectives. The platform successfully demonstrates that adaptive, QoE-driven content delivery and offline-first data management can be combined in a single, deployable mobile e-learning product. Chapter Five concludes the dissertation with a summary of findings, the project's contribution, recommendations, difficulties encountered, and directions for further work.

CHAPTER FIVE

CONCLUSION AND FURTHER WORKS

1. Summary of Findings

This project set out to design and implement Adaptis, an adaptive, network-aware mobile e-learning platform for students in low-connectivity environments such as Cameroon. A survey of fifty-nine prospective users confirmed the severity of the problem motivating the project: 95% of respondents reported poor or average internet connectivity while learning, 97% encountered recurring network problems, and 90% had abandoned an online lesson entirely because of a failed connection. These findings directly shaped a set of fifty prioritised functional and non-functional requirements, which were in turn translated into a formal system design covering context, data-flow, use-case, sequence, class and deployment diagrams, and a MongoDB-based data model built around a three-tier adaptive content structure.

The resulting platform was implemented as a Flutter mobile application, a React web dashboard for teachers and administrators, and a Node.js/Express/MongoDB backend exposing a JWT-secured REST API. Testing after deployment confirmed that user registration, authentication, navigation, course access and cross-platform responsiveness function as designed, and that the core adaptive-delivery and offline-synchronisation mechanisms operate correctly across the three content tiers. Taken together, these results demonstrate that a QoE-driven, offline-first architecture is a practical and effective response to the connectivity barriers identified at the outset of the project.

2. Contribution to Engineering and Technology

Where existing platforms such as Kolibri, Eneza Education and uLesson each solve one part of the connectivity problem in isolation - offline packaging, lightweight text delivery, or pre-loaded video, respectively - Adaptis contributes a single, integrated architecture that combines live, client-side Quality-of-Experience-based tier switching with an offline-first data model and automatic background synchronisation, applied specifically to the education domain rather than to entertainment streaming, where most existing QoE research is concentrated. At the implementation level, the project contributes a concrete, documented reference design - a Lesson data model that stores video, audio and text representations of the same content side by side, and a client-side decision engine that switches between them without a server round trip - that can be reused or adapted for other connectivity-constrained domains, such as mobile health

information systems or agricultural extension services, that face comparable infrastructural limitations to those documented for education in Cameroon.

3. Recommendations

- Institutions deploying e-learning platforms in low-connectivity regions should treat adaptive, tiered content delivery as a core requirement rather than an optional enhancement, given how consistently network instability was identified as the primary barrier to learning continuity in this study.
- Future development teams undertaking similar projects should budget explicit time for obtaining Google Play and Apple Developer accounts early in the project timeline, since store publication - rather than sideloaded distribution - materially affects a platform's real-world reach and credibility.
- Formal performance testing under simulated 2G/3G network conditions, rather than the manual network-tier simulation used during development, should be conducted before any wider release, to validate the bandwidth thresholds underlying the Good/Fair/Poor classification.
- A structured pilot test involving ten to twenty real students and teachers from the target user group, as originally planned during requirements validation, should be carried out to confirm that the adaptive behaviour matches expectations under genuine field conditions rather than only the simulated conditions used for the demonstration described in this dissertation.

4. Difficulties Encountered

Several difficulties were encountered over the course of the project. During deployment, the team encountered configuration issues and environment-variable mismatches between the development and production environments, along with minor runtime bugs that surfaced only after deployment and required targeted fixes. The absence of Google Play and Apple Developer accounts meant the mobile application could not be published to an official app store within the project timeline, and had to be distributed through Expo instead. Device-specific testing was complicated by the range of Android versions in use among prospective users, and several requirements - including the exact bandwidth thresholds separating Good, Fair and Poor connections, and the scope of the planned multilingual (English/French) support - remained only partially specified at the point of implementation and had to be resolved pragmatically by the team as development progressed.

5. Further Works

- Publish the mobile application to the Google Play Store and, subsequently, the Apple App Store, once developer accounts are secured.
- Conduct a structured pilot test with real students and teachers, and use the resulting data to validate and, where necessary, recalibrate the numerical bandwidth thresholds underlying the adaptive-tier classification.
- Complete the Should Have features that remain partially implemented, including real-time push notifications and the full teacher-student messaging workflow.
- Extend the platform with English/French bilingual support, scoped initially to interface strings rather than teacher-authored course content.
- Strengthen platform security and conduct a formal security review ahead of any production release.
- Expand the administrator analytics dashboard with deeper Quality-of-Experience reporting, to give institutions clearer visibility into how network conditions are affecting their students over time.
- Investigate carefully-scoped artificial-intelligence-based content recommendations, deferred during this version's requirements prioritisation, as a longer-term enhancement once the core adaptive pipeline has been validated in the field.

6. Conclusion

Adaptis demonstrates that unreliable connectivity need not be an insurmountable barrier to online education. By treating intermittent, low-bandwidth connectivity as the default operating condition rather than an exception, and by combining real-time network-aware content adaptation with an offline-first data architecture, the platform delivered in this project offers a practical, working answer to a problem that 97% of the students, teachers and prospective users surveyed for this study identified as a genuine, recurring obstacle to their learning. With the improvements identified above, the platform is well positioned to move from an academic prototype toward a solution capable of benefiting students across Cameroon and similarly connectivity-constrained regions.

REFERENCES

The list below was compiled from the technologies, standards and concepts referenced throughout this dissertation. It should be reviewed, verified and supplemented by the project team with any additional academic sources, textbooks or articles actually consulted during the literature review and design phases, so that every in-text citation is fully and accurately supported before final submission.

Apple Inc. (2023). HTTP Live Streaming (HLS) authoring specification. Apple Developer Documentation. <https://developer.apple.com/streaming/>

Bootstrap contributors. (2023). Responsive web design fundamentals. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Responsive_Design

Express.js contributors. (2023). Express.js: Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com/>

Fowler, M. (2005). Presentation Model. [martinfowler.com. https://martinfowler.com/eaaDev/PresentationModel.html](https://martinfowler.com/eaaDev/PresentationModel.html)

Google. (2023). Offline-first web application design. [web.dev. https://web.dev/learn/pwa/offline-data](https://web.dev/learn/pwa/offline-data)

GSMA. (2023). The Mobile Economy: Sub-Saharan Africa 2023. GSM Association.

ITU-T. (2017). Recommendation P.10/G.100: Vocabulary for performance, quality of service and quality of experience. International Telecommunication Union.

JSON Web Token contributors. (2023). Introduction to JSON Web Tokens. <https://jwt.io/introduction>

Krasner, G. E., & Pope, S. T. (1988). A description of the Model-View-Controller user interface paradigm in the Smalltalk-80 system. *Journal of Object-Oriented Programming*, 1(3), 26-49.

Laghari, K. U. R., & Connelly, K. (2012). Toward total quality of experience: A QoE model in a communication ecosystem. *IEEE Communications Magazine*, 50(4), 58-65.

Microsoft. (2023). The Model-View-ViewModel (MVVM) pattern. Microsoft Learn. <https://learn.microsoft.com/dotnet/architecture/maui/mvvm>

- MongoDB Inc. (2023). MongoDB documentation: Document data model.
<https://www.mongodb.com/docs/manual/core/document/>
- Mongoose contributors. (2023). Mongoose: Elegant MongoDB object modeling for Node.js.
<https://mongoosejs.com/docs/guide.html>
- Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. Proceedings of the USENIX Annual Technical Conference.
- React Native contributors / Meta. (2023). React: A JavaScript library for building user interfaces. <https://react.dev/>
- Sacha Greif & Michael Lynch. (2023). Flutter documentation: Building adaptive apps. Google.
<https://docs.flutter.dev/>
- Stellman, A., & Greene, J. (2014). Learning Agile: Understanding Scrum, XP, Lean, and Kanban. O'Reilly Media.
- Wieggers, K., & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press.
- World Bank. (2023). Digital Development in Sub-Saharan Africa. World Bank Group.

APPENDICES

Appendix A: Deployment Access Details

- Student mobile application - accessed via Expo Go, QR code, or the deployment link:
<https://flutter-node-stack--calvintando222.replit.app/>
- Admin Portal - <https://asset-manager--fluideowner.replit.app> (demo login:
admin@adaptis.edu / Admin123!)
- Teacher Portal - <https://teacher-portfolio--employeefluide.replit.app> (demo login:
sarah@adaptis.io / password123)

Appendix B: Survey Instrument

The full twenty-one-question stakeholder validation questionnaire referenced in Chapter Three can be accessed in the repository.

Appendix C: Selected Source Code Listings

Representative source-code excerpts (for example, the Lesson Mongoose schema and the JWT authentication middleware described in Chapter Four) should be attached here to support the implementation described in the main text. Or could be found in the folder repo.

Appendix D: Demo Videos and Final Presentation.

Find attached to this report a folder containing the various demo videos and the final presentation pptx file.